

Building A Knowledge Graph of a Music Collection

A case study in building Knowledge Graphs from Data Structures

16 min read · 3 days ago



Mark Burgess

Follow



Listen



Share

... More

In an earlier post ([Unifying Data Structures and Knowledge Graphs :Why we get confused about the difference between data and knowledge](#)). I drew attention to the distinction between “data” (detail oriented capture) and “knowledge” which is a much more personal assessment of a relationship to some data. I pointed out that data and knowledge are not the same thing, and that—in general—it’s a mistake to graph data sets—but there are some edge cases, where the overlap is strong. This follow-up article takes on a simple but hopefully familiar example that walks the line between data and knowledge for a beloved inventory. I’ll show how we can take some important data (such as an extensive catalogue of music files, or books) and turn them into "knowables" using the power of the Semantic Spacetime method.

Knowledge from data

Knowledge is about our *familiarity* with knowable material. So, if you want to know something, then a great way to start is to become a fan of the material. Music is a fine example, because we are all fans of our favourite music, and — as a result — we remember music better than a lot of other facts that we care less about. Moreover, we want to *really* learn about it, and also be able to find new details or simply recall facts that we can’t fully recall, despite knowing some aspects of the music like a friend.

We would love to have a knowledge base that we can add our notes and research to over time to build up

explorable stories about different artists, composers, and their lives.

The industry's answer to musical catalogues is to publish lists of albums or tracks, and to add “metadata” to the actual data of the music, which is embedded into music samples. The trouble is that, unless you only listen to the most basic kinds of pop music, the whole thing is a mess. Musical metadata standardization is in a pretty grim state of affairs. Allow me a small digression to explain.

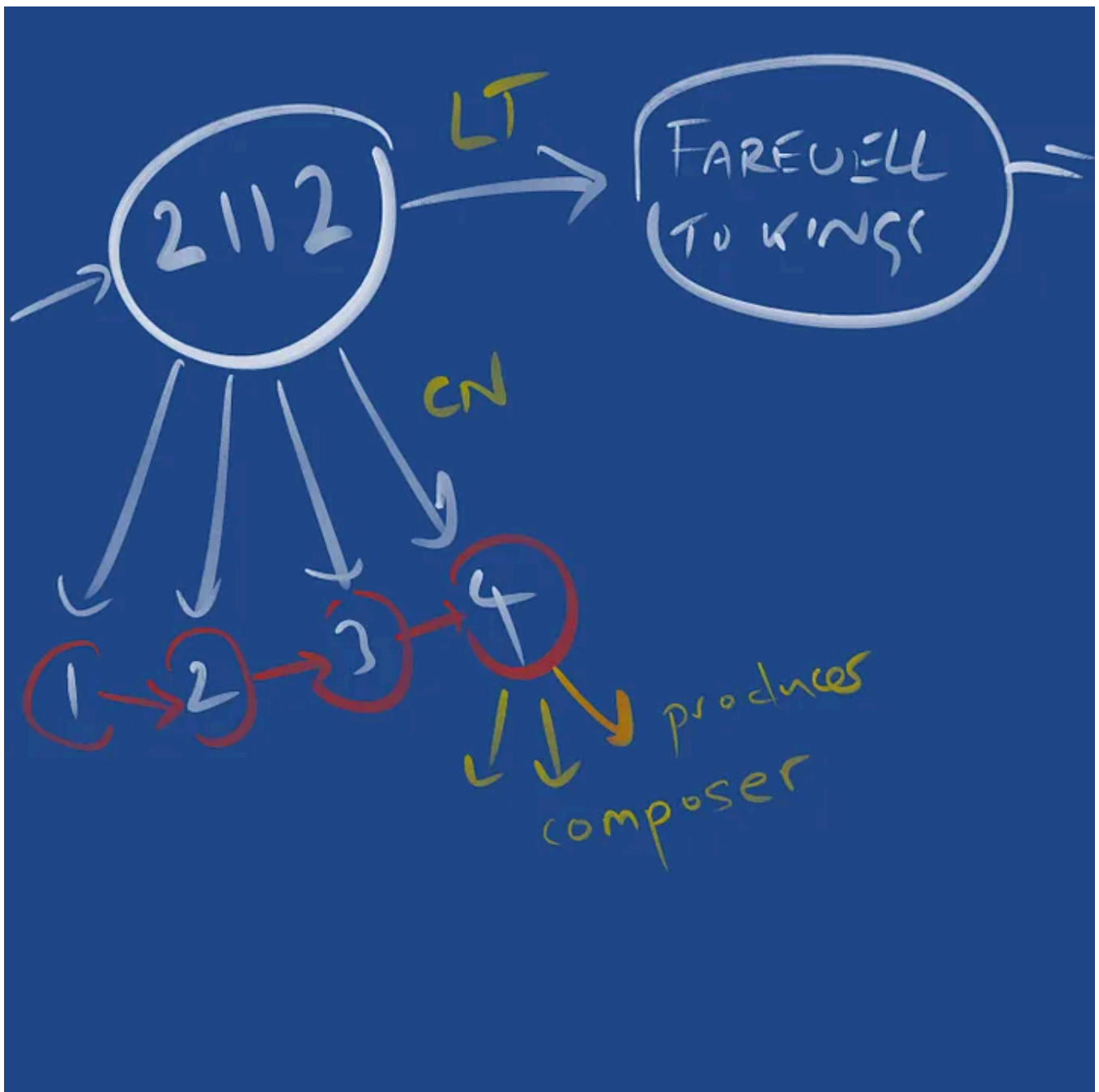


Fig 1: The "geometry" of a record collection (virtual model spacetime). It's sequences of albums containing sequences of tracks. Some things, but not everything is ordered. There are dates, durations, writing credits, etc. The basic structure has this "shape", but as many different "types" of information as we care to imagine.

When digital music was first invented, a simple schema was used to embed information about the *starts, stop, and durations*, and eventually the *titles* of tracks. Online databases, with their own schemas, were then maintained by music publishers so that you could “rip” the digital music into files, with software like **dBpoweramp** and friends, and then download the track listings and album covers etc.

Sometimes the metadata were actually wrong. A bigger problem was that the schema used to try to explain all music was based on the simplest kind of pop music release: every track has an artist and a title, as if that covered even the simplest production process. The industry settled on this model before it had really thought it through, in a rush to get music into shops on the new CDs. When they tried to shoehorn classical music, jazz, and other more sophisticated models into the artist-title model things got weird fairly quickly.

I consider every type of music to be my friend. So, now imagine a pretty ordinary classical music album, which contains two symphonies: the first is *Beethoven's 5th symphony*, played by the *London Philharmonic Orchestra* (LPO), conducted by *Fred WaveStick*. Who is the artist? Is it Beethoven, The LPO, or Fred? It depends who you happen to be cheering for at that moment. But there is only one space for the artist in the metadata.

The second symphony is a symphony by Vaughan Williams (or is it Vaughan-Williams hyphenated?) and it's also the 5th symphony, with the same conductor but a different orchestra! Where do we put this information? Do we squeeze it into a title, or into the artist? You will find every variation in practice.

The result is an utter mess, take it or leave it. The result is almost impossible to search. Music streaming companies have done their best to provide searches that allow you to find what you're looking for, but only if you know enough about it. In the streaming age we've lost the valuable album notes or CD booklets, which typically contained accurate credits and information about composers, lyrics, and so on.

Digital music was designed for singles. It has no data structure for albums, only for single tracks!! One has to infer the existence of an album by looking at the information on all of the tracks.

Although the model has been extended a little, what actually happens is that each music release tried to stuff all the information into the few lines, so that the title says something like: “*Beethoven and Vaughan-Williams: A Perfect Fifth*” and the artist field says “London Philharmonic Orchestra (conducted by Fred WaveStick) and The Royal Concertgebouw Orchestra (conducted by F.A. WaveStick)”.

Today, we have digital files (*.mp3, *.flac, *.m4a etc), which contain the metadata embedded. We can extract it and try to organize it. Here are some examples: from the album by rock band “Rush”, we have a track:

Fig 2: Metadata from a music file

Notice that the album artist is blank, the artist is Rush, and the composer is Rush. As a fan of Rush, I happen to know that the band all collaborated on the music and that the lyrics were written by Neil Peart. On the original vinyl release (and later CD) they would credit like this: Music by Alex Lifeson and Geddy Lee, and Lyrics by Neil Peart. This is something, which is interesting to capture, and I would like to correct the metadata manually in my searchable knowledge base! Here are two more examples:

Fig 3: Metadata from two other tracks

Notice that the year of the first is botched. It should read 1977. If you were going to put this information into a database—even the most basic kind of catalogue—then you would typically want all of these names and titles to be searchable by putting them into different fields in a schema. But you don’t know the full schema, or how many entries “composer” or “orchestra” should have.

Modern “schemaless” databases can deal with these issues—and *graph databases* are very well suited, because — in a graph — it doesn’t matter how many links you have to (say) composer from a particular title, whereas in a hierarchical structure, like JSON, you can only make lists. In a graph, arrows from A to B allow you to know that "*B composed A*", and find all the "*As that B composed*", by following the arrows in two directions.

So, with this digressive complaint in mind, let’s consider how to model music as a semantic spacetime, using the [SSTorytime project](#) and the geometry sketched in figure 1.

The first step is to extract the different independent meanings from this potpourri of metadata strings.

From summarization of atoms to research

The atoms of music are now “tracks”. These were once taped together on an analogue recording, then separated by wider spacing of grooves on vinyl; then they were encoded with digital protocols on CD. Now they are independent files on a disk.

Planning out the structure of a graph is quite hard to think about — it feels abstract and the API for generating it does nothing to help. As humans, we are much better at thinking in terms of notes on a page, lists, etc. This is where the SSTorytime [note language N4L](#) is helpful.

Logic based knowledge graphs (RDF/OWL) etc are optimized for logical expression evaluation, but Semantic Spacetime is optimized for *explainable storytelling*. Making a tool analogous to [text2N4L](#), but for scanning music files is straightforward. Let’s call it **music2N4L**. It will extract what there is of metadata from the files, as above, and try to digest it into some basic arrow annotations and relationships.

Music is usually stored with one directory of files per "commercial release package". These are stored under master directories, labelled by the artists, so we can walk

this tree of files and pull out the data. The directory structure is only for tidiness: it contains no information of value in itself. Moreover, we can't assume that the files appear in the order that represents their order on the album.

A directory or catalogue is a list of entries. It's perhaps the one case in which data and “knowables” are very close to one another, but there are still differences. For example, the data are sourced from the atomic files, but that information is—on the one hand—both hopelessly incomplete and—on the other—also *redundantly replicated* in every track belonging to every album or release. Because the entire album information has to be inside every track, we have to assemble an album from its constituent tracks and then try to make sense of the collection. Even album covers can be embedded in the music tracks today.

Our goal then, is to generate a set of editable N4L notes which expresses a basic graph that we can later edit for completeness. If we simply laid out all of the information for every track, we would scarcely be able to see where one album ended and the next began. The order would be mangled (file order is random, so track number is encoded explicitly). Also, there would be lots of redundant or repeated information. It would be very hard to read, and we still have the problem of extracting the relevant categories of information (roles or artists, performers, orchestras etc) from the few strings that are ignominiously stuffed into the files like a thankless Turkey.

The answer, like all knowledge, is to get to know how the data work—by looking at many examples and seeing what we want to try to do with it. We would like to extract common data and attach these as annotations to the album or “release container”, which has its own title name.

A semantic spacetime knowledge graph can help by introducing links with names that map to the simple attributes, like composer, conductor, band, soloist, etc. These, by design, must fall into one of four generic types: either about successive ordering (LEADS TO), being part of a whole (songs on an album, movements in a symphony, etc (CONTAINS), about attributes like the producer (EXPRESS PROPERTY), or about closeness/similarity i.e. equivalences, like Vaughan Williams vs Vaughan-Williams or Borodin vs Бородин (NEAR/SIMILAR). No matter the specific labels we use, the way we follow these different kinds of information is always the same. What we need most is not an ontology, but a geometry.

So we have to think about the kinds of stories we want to tell about the music (figure 1). Here are some point to start with:

- We want to be able to preserve the order or tracks on an album (at least, if you're old enough to care about albums). For this, we can use the sequencing feature of N4L to simplify matters.
- We may also want to preserve the order of albums in a sequence (e.g. multiple LP or CD boxes, or release histories that chart a band's evolution).
- We may want to follow the careers of conductors, orchestras, band members, producers, etc.
- We certainly want to know which tracks belong to which albums—though this is not necessarily simply hierarchical as tracks can be included in music compilations.
- Today people may want to encode playlists of unrelated material for various occasions, or someone who uses music in film might want to classify by mood, inspiration etc (a special LEADSTO sequence).

Music is a great example of how knowables are things that *matter* to us, and that we *intend to know*.

So the kind of graph we want will look something like this: a number of genre-separated sequences of albums, each containing a sequence of individual tracks (see figure 1). So basically, a specialist music shop.

Fig 4: What it used to look like (real spacetime)

It has containers of parts, joined together into chains of events. Musical performances and releases are “events” in the semantic spacetime model, not just “things” (figure 1).

Conquering the files

The first step is to set up N4L. This is one that can continue to evolve as we get to know the data. All notes in N4L have the form:

text 1 (arrow) text 2 (arrow2) text3 ...

We go first to the SSTconfig directory, of the directory we are editing and define the kinds of arrows we want. They are separated into four files for the four kinds of arrow. Here are the ones I put in the oven earlier:

Fig 5: NEAR type arrow definitions we might make. The parenthetic short forms are short aliases that can be used when editing, for convenience.

As we've seen, artist names are sometimes written in different ways, so we'll want to link up different spellings. We could normalize them, or preserve the discrepancies as valued information but still point out that they are intended as synonyms ("also called"). Similarly, we might need to disambiguate names that are the same but which refer to different people. Musical genres are a mess too. "Classical" is the same as "Classique" is like "Symphonic", with various languages in play. Cinematic film soundtracks have the most hilarious variation in genres attribution, revealing the utter cluelessness of music publishers. Each album release may have a different description of the genre from "soundtrack" to "Bandes originales de films".

Fig 6: Some basic LEADS TO type arrow definitions. Notice that directional arrows have inverse meanings too, as the — for every +.

Using the N4L sequence mode to manage an ordered list, tracks can be sorted and ordered by music2N4L and added to a “:: _sequence_ ::”, which results in links

ordering by the default “then” relation within each album. So, for connected albums, we still need one more “next album” for album sequence order. We will likely come back to add other later.

Fig 7: Some CONTAINS arrow definitions

For containment, we’ll want to think in terms of different groupings.

- Track inside an album,
- Albums inside collections,
- Either inside genres, etc.

These names can all be chosen as we wish. The only important distinction is that they are part of the “contains” arrows.

Then finally, property attribute naming is the largest group of distinctions to make. This list might be excessive, but a fan quickly gets ideas to implement!

Fig 8: A few EXPRESS PROPERTY arrows we might need...

This list will doubtless grow as we find more interesting facts to add from external research.

With the help of a simple software tool music2N4L we extract the parts of the text and generate a file, which looks like something like this:

Fig 10: The basic output of music2N4L on a single album. There's a lot that can't be inferred from the metadata, but it's not hard for a human to fix.

Sometimes, there's enough information to make a direct identification, other times it defaults to indecipherable.

Some metadata could fit into a simple kind of traditional fixed schema, but there are also repeated items and comments that we may want to add ad hoc. Who is this conductor and what do we know about him? Who were the members of the rock band, did they have any solo albums? Were they in several bands over the years? These notes have to come from independent research (or by searching for the mythical album notes), and can be added over time as a labour of love.

After some editing, we rearrange, delete and tidy this manually:

Fig 11: Edited album information in N4L — which can now be compiled directly

Notice that there are two soloists who didn't make it into the metadata, but were listed on the album cover! This is par for the course in music releases.

We can continue to embellish the form of each track and album, and also start to document the actual people involved, their lives and their accomplishments. Once the basic data skeleton is in place, it's entirely a human operation to make this into a valued knowledge store.

The N4L context syntax:

```
:: classical, orchestral ::  
blabla  
  
:: film music, cinema, tv ::  
blulblu
```

is used to collate entries of similar intent. This is a simple way of distinguishing genre by a more exacting standard than the (often ridiculous) official classifications. Here, we simply move entries around to group them together under common

headings. Contexts can overlap freely, they are not taxonomic binary trees, but more like tags.

Uploading for searching

Notes are how we plan and execute the editing of the material. Then we want to be able to search and explore. The final result of massaging the data, for several albums, looks like [this example](#)- To upload to a database, we compile the edited file:

```
N4L -u example_collection.n4l
```

We are now free to search the information cached in the resulting graph database, either using the default browser interface or by creating something more specialized by scripting a particular application:

Fig 12: the local graph of nodes referring to the search “Williams”

Fig 13: The local orbits of a particular album. Indentation shows distance from the title item.

With that minimum of effort to edit the notes using familiar text editing tools, we can make sense of a search about a person:

Fig 14: The local orbits of Mark Stone

Had we made arrows to distinguish between summary of soloists on album and soloists specifically by track (or some such names), this would also show the difference explicitly, and we could restrict the search to show only one or the other—but it's easy enough to make that inference as a human browser.

We can also follow different views, to look for story lines or simply browse the notes:

Fig 15: The shape of the notes we made from the example

Fig 16: The rendering of the notes directly from the database cache

Fig 17: A sequence view of the album 2112

Notice that, at no point, did we have to think about logical reasoning, or ontological significance. At best, we used intuitive relationships — some of them (such as “properties”) look just like a schema relationship column names, while others look like partial ordering relations. All we need to do is think about: is it information about "space or time"? The graph follows with the help of the N4L compiler.

The resulting graph is similar to the extensive list of metadata, but it has been massaged, reduced, edited, and merged with personal research. The original data is in no way a substitute for the curated graph of “knowable” notes we finally end up with.

Why People Get Knowledge Representation and Reasoning Wrong

There are many people trying to build knowledge graphs in the image of databases. What I mean by this is: simply enumerating every piece of data exhaustively for logical correctness and expression evaluation rather than readability. It typically means using *typed data* instead of causally structured notes to try to navigate meaning.

Types are superfluous in most cases if you choose link arrow names sensibly. Arrows effectively induce types on nodes and work as flexible substitutes.

The RDF/OWL/ontology tradition was designed by engineers using the formative ideas of network computing thirty years ago. But after 30 years, it ought to be clear that these methods are cumbersome, hopelessly brittle, and little better than any other database. Graphs, without a methodology, are simply schema-less databases like many others. If a schema-less graph contains nothing more than a partially full scheme, what have we accomplished by using it?

Knowledge is the domain of teachers and writers, but we have listened to technologists playing bureaucrats when designing data stores. Our models of “knowledge representation” have been based around tediously brittle type systems, or machine schemas renamed as ontologies—basically, applying the methods of exhaustive bookkeeping/accounting to a pursuit that ought to be creative.

Bookkeeping, data forms, and other tools of bureaucracy were methods of mechanization from an age before modern machines were invented. They constrain humans to act like machinery, rather than automating and freeing humans to steer the overall process. They sucks the life out of the process and are a sure way to kill someone’s interest before any real knowables get captured.

What SST offers is a form of story inference by causal coherence.

The dimensional similarities of stories are something like the semantic equivalent of the units of dimensional analysis in physics.

It's important for a curator of knowledge to feel a freedom to write what's on their mind. If we set up barriers and obstacles through ill-placed formality, it's only self-sabotaging. This is also why developers often fail to make progress: focusing on details and future obstacles, trying for one-shot perfection rather than staged progress.

Knowledge is not a system of record for data

A semantic database is not generally a system of record: it's not a museum of artefacts for scientific scrutiny, not a perfect image frozen for the purpose of preserving raw data, rather it's a supplementary index and an interpretational narrative. It can still point to those frozen raw artefacts as part of its links.

The value of semantic spacetime, and its linkage to note-taking, is to rebuild a geometry for browsing from notes that are the real value—notes are our system of record.

A map of knowables should represent our best understanding of our notes on a given day. It's relatively quick to regenerate the graph from the notes, and relatively easy to use familiar tools to edit the notes as text, so there's no reason not to keep it up to date. There are plenty of techniques for managing the update process. As a fan, this becomes a labour of love. In order for material to become knowledge, that is the process to invest the time in.

End note

There will always be someone who gravitates to a logical view. It's not without its uses. If this is important, you can choose to gradually move towards greater formality once you have something working, i.e. producing useful results. Too many groups will try to implement a massive project, steeped in inappropriate constraints, without knowing that the outcome will be worthwhile. Ironically, reasoning is easier without the formal logic — but that's a story I've told elsewhere.

Once you understand the semantics that are important to you, you can dabble with types. Until then, they will only be your enemy. And, as for data — they can form the backbone of your knowledge, but to flesh out something meaningful takes human input. So with that in mind, let the show begin.

Knowledge Graph

Knowledge

Semantic Spacetime

Knowledge Management

Storytelling



Follow

Written by Mark Burgess

990 followers · 2 following

@markburgess_osl on Twitter and Instagram. Science, research, technology advisor and author - see [Http://markburgess.org](http://markburgess.org) and [Https://chitek-i.org](https://chitek-i.org).

Responses (2)



Bgerby

What are your thoughts?

See all responses

